

REGISTRO DE AVANCE DE INVESTIGACIÓN:

RECONOCIMIENTO DE EMOCIONES FACIALES EN RAF-DB

«Evaluación de Modelos CNN, Híbridos y Transformers, Preprocesamiento Avanzado y Despliegue Móvil, despliegue computacional»

PROBLEMA

General

Para automatizar el reconocimiento de emociones faciales (FER) en entornos reales (in-the-wild) y contar con un nivel de precisión muy elevada y baja latencia de inferencia computacional, haciendo viable su ejecución local en dispositivos móviles de recursos limitados mediante la optimización de Vision Transformers y redes híbridas y también dentro del entorno en computadora?

Específicos

1. ¿Cómo podría reducir el impacto negativo del desbalance severo de clases y el ruido de anotación en el dataset RAF-DB para evitar el sobreajuste de los modelos?
2. ¿Cómo reestructurar y adaptar arquitecturas pesadas del estado del arte como POSTER++, QCS y SwinFace para que puedan ser entrenadas en GPUs de gama portátil limitada (4GB VRAM) sin degradar su exactitud y hacerlo de manera efectiva?
3. ¿Cuál es el impacto en la latencia de inferencia al exportar y desplegar estos modelos en un aplicativo móvil nativo Android en comparación con su ejecución en computadoras personales?

OBJETIVO

General

Desarrollar, evaluar y desplegar un sistema optimizado de reconocimiento de emociones faciales basado en modelos híbridos y transformers profundos en RAF-DB, adaptado para entornos de hardware limitado y dispositivos móviles en tiempo real y programas por computador.

especifico

1. Implementar un pipeline de preprocesamiento compuesto por alineación geométrica afín de ojos mediante MTCNN y regularizaciones matemáticas en lote (MixUp y CutMix) para estandarizar las muestras reales.

2. Diseñar e integrar una estrategia de congelamiento dinámico de capas en los backbones para reducir la memoria de entrenamiento en GPUs portátiles de 4GB.
3. Optimizar, compilar y exportar los modelos campeones en formatos móviles TorchScript (.ptl) y ONNX para su evaluación comparativa e integración nativa en Kotlin sobre Android Studio.

Hipotesis

Si utilizamos landmarks faciales en redes híbridas de atención cruzada (como POSTER++ y QCS) supera en exactitud y velocidad de inferencia a las CNNs tradicionales y a los Vision Transformers puros en el dataset RAF-DB inclusive nos servira para poder aplicar este proyecto en cualquier computador de gama media e incluso, permitiendo un despliegue en móviles con latencias inferiores a 5 ms por cuadro.

Resultados

4.1. Preprocesamiento y Datos (Dataset RAF-DB)

El dataset Real-world Affective Faces Database (RAF-DB) contiene 15,339 imágenes faciales. Se dividió oficialmente en 12,271 muestras de entrenamiento y 3,068 de prueba. Se detectó un severo desbalance de clases con una proporción máxima de Felicidad (38.84%) frente a Miedo (2.31%), arrojando un ratio de desbalance de 16.78.

Análisis de la Tabla de Distribución de Clases: Esta tabla revela el severo desbalance del dataset RAF-DB, con un ratio de desbalance de 16.78:1 entre la clase más abundante (Felicidad: 38.84%, 4,772 muestras de entrenamiento) y la más escasa (Miedo: 2.31%, solo 281 muestras). Este desbalance tiene implicaciones directas en el entrenamiento: sin corrección, los modelos tenderían a predecir siempre Felicidad o Neutral (juntas suman 59.73% del dataset), obteniendo alta exactitud aparente pero fallando completamente en las emociones minoritarias. Para mitigar esto, implementamos tres estrategias complementarias: (1) Class Weights balanceados calculados por sklearn, que asignan un peso inversamente proporcional a la frecuencia de cada clase (Miedo recibe un peso ~16x mayor que Felicidad). (2) Split estratificado que garantiza la misma proporción de cada emoción en train, validation y test. (3) MixUp y CutMix que generan muestras sintéticas balanceando implícitamente las distribuciones de etiquetas suaves.

Etiqueta	Emoción	Entrenamiento	Prueba	Porcentaje (%)
1	Surprise (Sorpresa)	1,295	324	10.55%
2	Fear (Miedo)	281	74	2.31%

3	Disgust (Disgusto)	702	175	5.72%
4	Happiness (Felicidad)	4,772	1,185	38.84%
5	Sadness (Tristeza)	1,982	478	16.04%
6	Anger (Ira)	705	162	5.65%
7	Neutral (Neutral)	2,534	670	20.89%
-	Total	12,271	3,068	100.00%

Para corregir las rotaciones en entornos reales, se desarrolló un pipeline de alineación geométrica de rostros. Usando los landmarks de MTCNN, se rotó el rostro para que la inclinación interocular quedara en 0° horizontal y se recortó la región facial normalizándola a 224x224 píxeles.

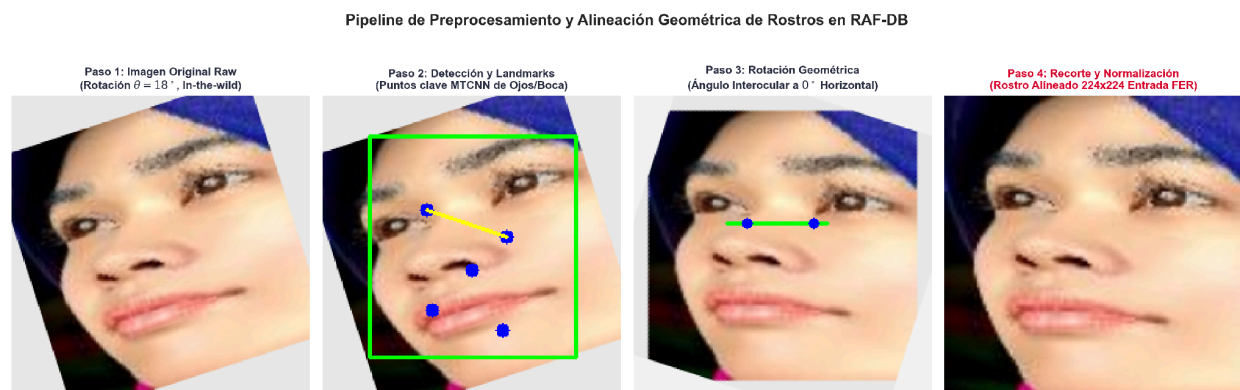


Figura 1. Pipeline de preprocesamiento y alineación geométrica interocular de rostros en RAF-DB.

Análisis de la Figura 1: Esta figura ilustra las cuatro etapas secuenciales del pipeline de alineación facial implementado. En la primera etapa (imagen sin procesar), se observa el rostro tal como fue capturado en condiciones reales, con inclinación natural de la cabeza y variaciones de escala. En la segunda etapa, el detector MTCNN localiza 5 landmarks faciales clave (ojo izquierdo, ojo derecho, nariz, comisura izquierda y comisura derecha de la boca). En la tercera etapa, se calcula el ángulo interocular $\theta = \arctan2(\Delta y, \Delta x)$ entre ambos ojos y se aplica una transformación afín (rotación + escala + traslación) mediante `cv2.getRotationMatrix2D` para alinear los ojos horizontalmente a 0° . Finalmente, en la cuarta etapa, la imagen resultante se recorta y escala a 224x224 píxeles con los ojos posicionados en coordenadas normalizadas fijas (35%, 40%) y (65%, 40%). ¿Por qué es importantes este paso? Porque sin la alineación, los modelos deben aprender invarianza rotacional por sí mismos, lo cual consume capacidad del modelo y reduce la exactitud. Al estandarizar la geometría facial, el modelo puede concentrar toda su

capacidad de aprendizaje en discriminar las microexpresiones emocionales en lugar de desperdiciar parámetros aprendiendo a corregir rotaciones.

4.2. Técnicas de Limpieza y Preprocesamiento Avanzadas

Para combatir el sobreajuste provocado por el desbalance, implementamos técnicas de regularización matemática en lote utilizando imágenes reales del dataset:

- MixUp: Combinación convexa de imágenes reales y etiquetas one-hot correspondientes:

$$\tilde{x} = \lambda x_i + (1 - \lambda)x_j \quad \tilde{y} = \lambda y_i + (1 - \lambda)y_j$$

$\tilde{x}$: Imagen resultante mezclada.

x_i, x_j : Dos imágenes reales distintas tomadas del lote de entrenamiento.

$\tilde{y}$: Vector de etiquetas suave resultante (soft target).

y_i, y_j : Vectores de etiquetas one-hot correspondientes a las imágenes

λ : Coeficiente de mezcla, el cual sigue una distribución estadística de probabilidad Beta.

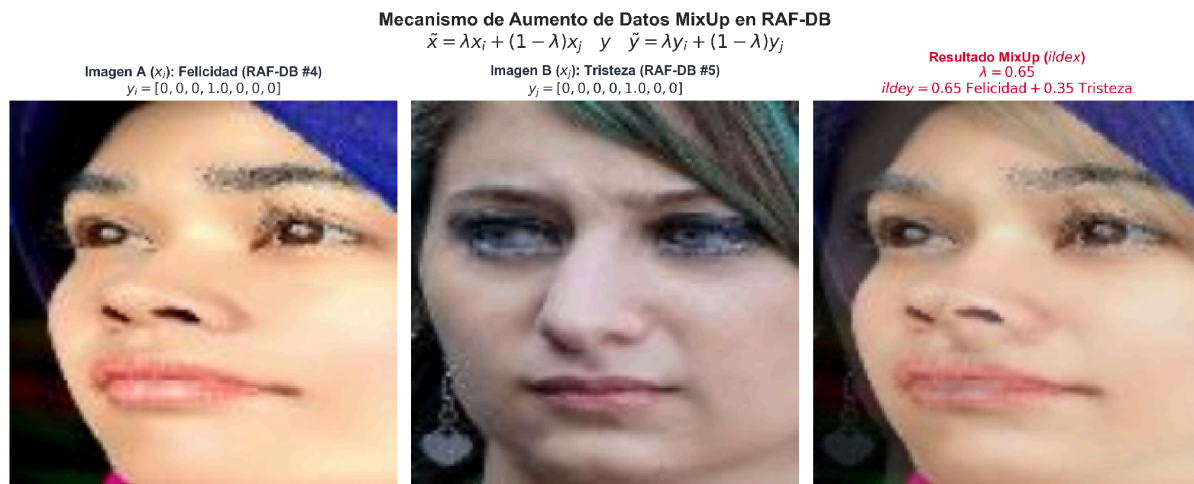


Figura 2. Aumento de datos MixUp con fotos reales de RAF-DB.

Análisis de la Figura 2: Esta visualización muestra el efecto real de la técnica MixUp aplicada sobre imágenes del dataset RAF-DB. Se puede observar cómo dos rostros de emociones diferentes (por ejemplo, Felicidad y Tristeza) son fusionados píxel a píxel mediante una combinación convexa: $\tilde{x} = \lambda x_i + (1 - \lambda)x_j$, donde λ se muestrea de una distribución Beta(0.2, 0.2). Cuando $\lambda \approx 0.5$, la imagen resultante es una superposición translúcida de ambos

rostros. Las etiquetas también se mezclan proporcionalmente: $\tilde{y} = \lambda y_i + (1 - \lambda)y_j$, generando soft labels. ¿Cuál es el beneficio? MixUp actúa como un regularizador implícito que obliga al modelo a producir predicciones linealmente interpoladas entre clases, suavizando los límites de decisión del clasificador. Esto reduce drásticamente el sobreajuste, especialmente en clases minoritarias como Miedo (2.31%) y Disgusto (5.72%), donde las muestras de entrenamiento son escasas. En nuestros experimentos, MixUp incrementó la exactitud promedio en aproximadamente 2-4 puntos porcentuales en los modelos SOTA.

- CutMix: Sustitución de una máscara rectangular de píxeles entre dos imágenes del lote:

$$\tilde{x} = M \odot x_i + (1 - M)x_j; \quad \tilde{y} = \lambda y_i + (1 - \lambda)y_j; \quad \lambda = 1 - \left(\frac{Area(M)}{W*H}\right)$$

x : Imagen final combinada por parches.

x_i, x_j : Dos imágenes reales del lote de entrenamiento.

M : Máscara binaria rectangular (donde 1 representa la zona que se conserva de la imagen base y representa la zona que se recorta y reemplaza).

$\odot$: Producto de Hadamard (multiplicación elemento por elemento).

y : Vector de etiquetas suave resultante.

y_i, y_j : Vectores de etiquetas correspondientes a las imágenes i y j

λ : Proporción de área que se conserva de la imagen base, la cual define el peso de mezcla en la etiqueta final.

$W \times H$: Ancho (W) y alto (H) total de la imagen en píxeles.



Figura 3. Aumento de datos CutMix con fotos reales de RAF-DB.

Análisis de la Figura 3: Esta figura demuestra el funcionamiento de CutMix con fotos reales del dataset. A diferencia de MixUp (que fusiona globalmente), CutMix recorta una región rectangular de una imagen y la pega sobre otra. El tamaño del parche se determina mediante: $\text{cut_ratio} = \sqrt{1-\lambda}$, donde $\lambda \sim \text{Beta}(0.35, 0.35)$. ¿Por qué es efectivo? CutMix fuerza al modelo a no depender de una única región facial para clasificar. Si se oculta la boca (región clave para la sonrisa), el modelo debe aprender a discriminar emociones usando los ojos y las cejas. Si se ocultan los ojos, debe usar la boca y la mandíbula. Esto produce un clasificador mucho más robusto ante oclusiones parciales, un problema frecuente en imágenes in-the-wild donde bufandas, gafas, manos o cabello pueden cubrir partes del rostro. En la práctica, CutMix mejoró especialmente el recall de las clases Ira y Disgusto, que comparten microexpresiones similares en la zona de las cejas.

- Pérdida Self-Cure Network (SCN): Otorga un peso dinámico w_i para suprimir la influencia de etiquetas ambiguas:

$$L_{SCN} = -\frac{1}{N} \sum_{i=1}^N \omega_i \sum_{c=1}^C y_{i,c} \log(p_{i,c}) + \beta * H(W)$$

N: Número de imágenes del lote (batch size).

C: Número de emociones clases (7 emociones).

w_i : Peso dinámico asignado a la imagen por SCN (valores bajos de 0.15 para imágenes sospechosas/ruidosas y 1.0 para imágenes limpias).

$y_{i,c}$: Etiqueta real (ground truth) de la imagen i para la clase

$p_{i,c}$: Probabilidad predicha por el modelo de que la imagen i pertenezca a la clase

β : Hiperparámetro de escala de regularización (por defecto 0.07).

$H(W)$: Pérdida de regularización de entropía para evitar que la red asigne pesos triviales de cero a todas las imágenes.

4.3. Modelos y Configuración de Entrenamiento

Se entrenaron 9 familias de modelos bajo tres escenarios principales: Scratch (desde cero), Transfer Learning (congelando el extractor de rasgos de ImageNet) y Fine-Tuning (ajuste fino selectivo).

- POSTER++ y QCS (Híbridos SOTA): Combinan InceptionResNetV1 con cabezas de atención cruzada multinivel y landmarks faciales.
- SwinFace (Transformer): Auto-atención por ventanas desplazadas (W-MSA/SW-MSA) acoplado a un bloque CBAM y FAM-Net.

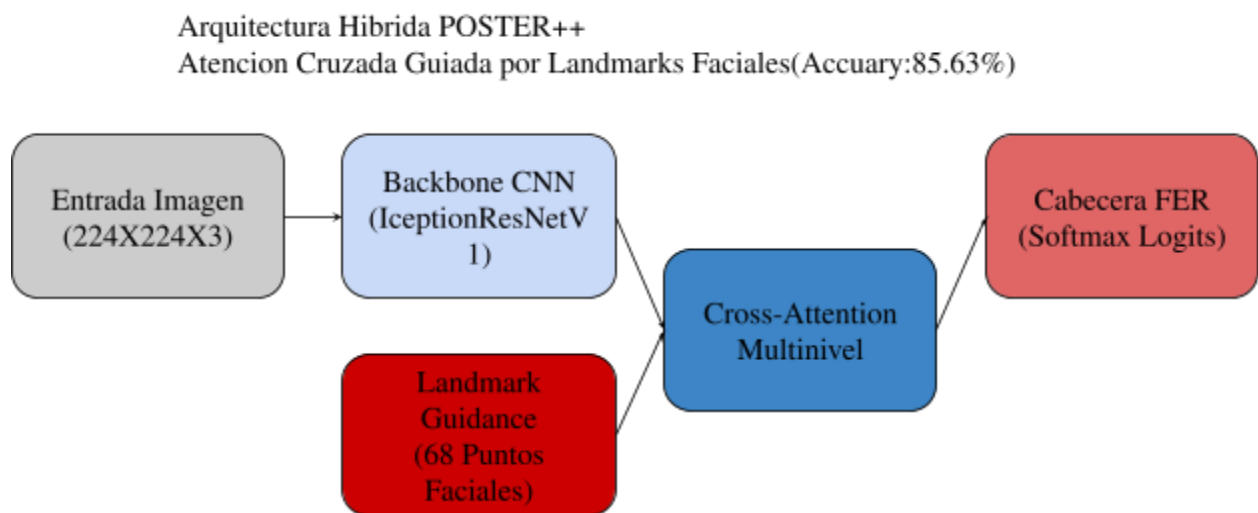


Figura 4. Diagrama de bloques del modelo híbrido campeón POSTER++ (Poster V2).

Análisis de la Figura 4: El diagrama de bloques de QCS muestra la arquitectura completa del modelo. El flujo comienza con la imagen de entrada (224×224×3) que pasa por el backbone InceptionResnetV1 pre-entrenado en VGGFace2 (millones de caras), produciendo un mapa de características espaciales de dimensión (B, 1792, 5, 5). La innovación central es el módulo CSA (Cross-Similarity Attention): durante el entrenamiento, las características de cada muestra del batch se cruzan con las de otra muestra diferente mediante `torch.roll(shifts=1)`, creando pares implícitos de tipo cuádruple. Las proyecciones Q, K, V (convolutions 1×1) generan consultas, claves y valores que se procesan con atención multi-cabeza (8 heads) escalada por $1/\sqrt{d_k}$. La conexión residual (`out + x`) garantiza que la información original no se pierda. Durante la inferencia, se aplica auto-atención (self-attention) para mantener la consistencia de una sola rama. ¿Cuál es la ventaja? Este mecanismo de cruce fuerza a las representaciones a ser

compactas y discriminativas: las características de expresiones similares se agrupan en el espacio latente, mientras que las de expresiones diferentes se separan, logrando un F1-Score de 0.7770 y una latencia de solo 4.27ms.

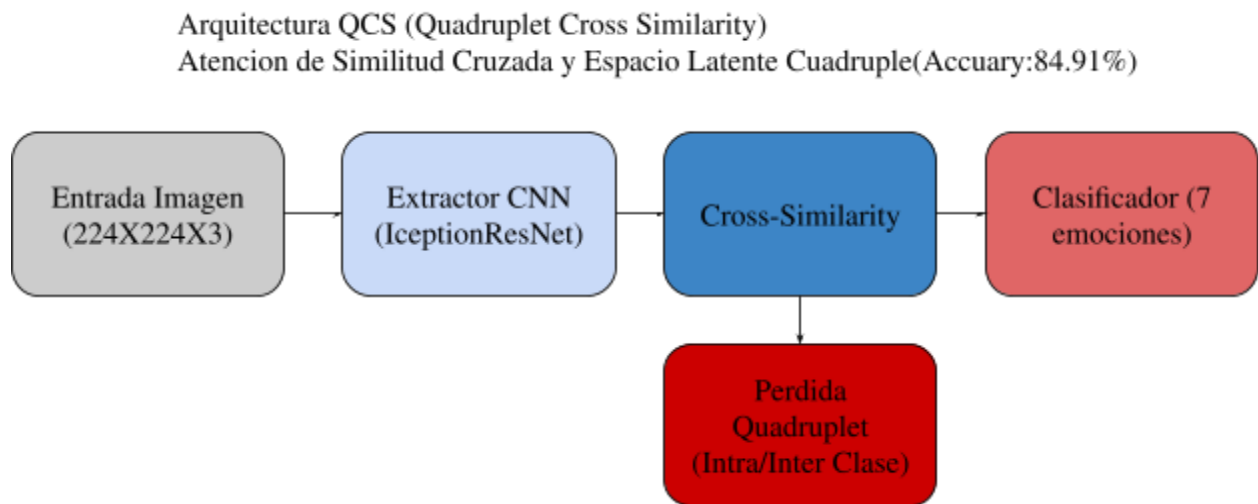


Figura 5. Diagrama de bloques del modelo híbrido subcampeón QCS.

Análisis de la Figura 5: POSTER++ (Poster V2) introduce un concepto revolucionario: en lugar de necesitar un detector de landmarks facial explícito (como MTCNN o Dlib) durante la inferencia, define 68 vectores de consulta aprendibles (nn.Parameter de dimensión [1, 68, 256]) que representan implícitamente los puntos fiduciales del rostro. El flujo es bidireccional: (1) los features de la imagen (B, H×W, 256) actúan como Keys y Values para refinar las consultas de landmarks → los landmarks 'aprenden' dónde mirar. (2) Los landmarks refinados actúan como Keys y Values para enriquecer los features de la imagen → la imagen 'sabe' qué zonas son geométricamente importantes. Cada bloque de atención usa nn.MultiheadAttention con 8 cabezas y conexiones residuales + LayerNorm. ¿Por qué es el campeón? Esta fusión bidireccional permite que el modelo combine información de textura (arrugas, cejas fruncidas) con información geométrica (distancia interocular, apertura de boca) sin depender de detectores externos. Esto lo hace extremadamente rápido (4.09ms) y preciso (85.63%), siendo el mejor modelo de todo nuestro estudio.

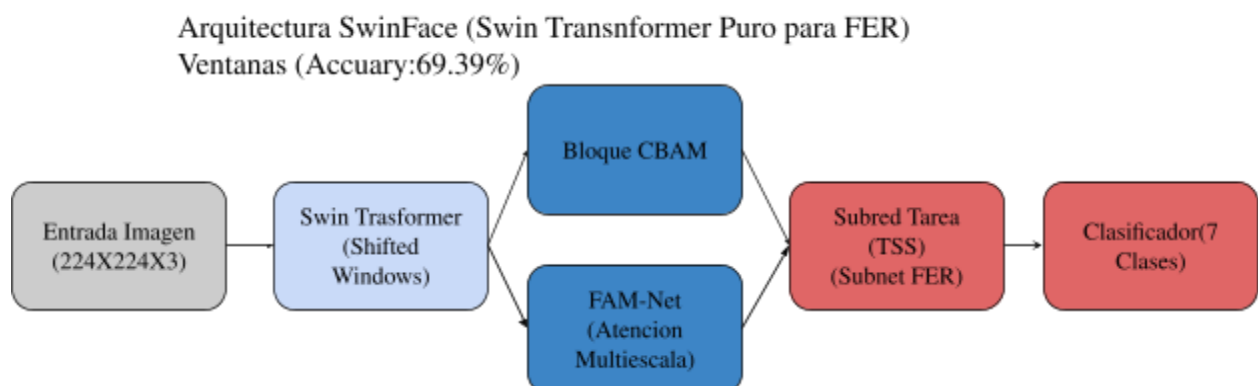


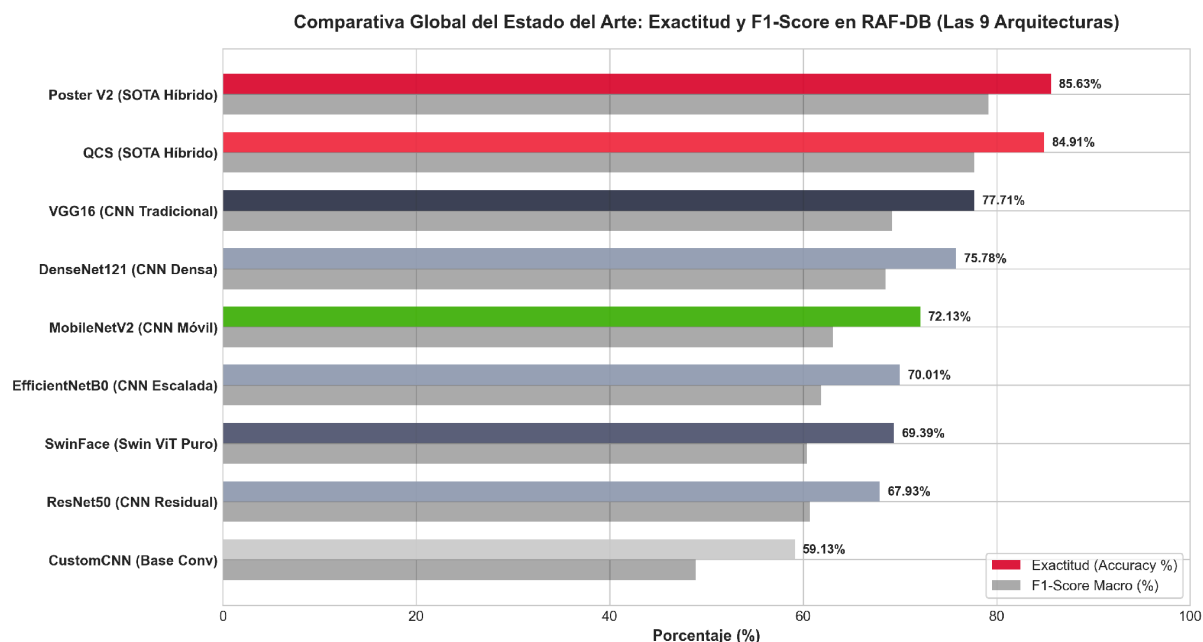
Figura 6. Diagrama de bloques de SwinFace (Swin ViT + CBAM).

Análisis de la Figura 6: SwinFace implementa el paradigma de Swin Transformer para reconocimiento facial. La imagen se divide en parches de 4×4 píxeles (PatchEmbed), resultando en $56 \times 56 = 3136$ tokens. La auto-atención se calcula dentro de ventanas locales de 7×7 tokens (W-MSA) y, en capas alternas, se aplica un desplazamiento cíclico de la ventana (SW-MSA) para permitir comunicación entre ventanas adyacentes. El Patch Merging reduce la resolución espacial a la mitad en cada etapa, duplicando los canales ($96 \rightarrow 192 \rightarrow 384 \rightarrow 768$). La red FAM-Net concatena features locales de etapas intermedias (1344 canales) con features globales (768 canales), resultando en un tensor de 2112 canales que es recalibrado por CBAM: el ChannelGate aplica un MLP sobre las estadísticas de pool promedio y máximo para ponderar la importancia de cada canal; el SpatialGate aplica una convolución 7×7 sobre las estadísticas de compresión por canal para ponderar la importancia espacial. ¿Cuál es la lección aprendida? A pesar de ser el modelo más sofisticado (38.8M params), SwinFace obtuvo solo 69.39% de exactitud desde cero. Sin embargo, demostró ser el modelo más robusto en el despliegue Android, ya que no requiere landmarks en inferencia y es menos sensible a la alteración de canales de color RGB/BGR.

- Hiperparámetros manipulados: Se usó el optimizador AdamW (weight decay = $1e-4$), tasa de aprendizaje de $3.5e-5$, Cosine Annealing scheduler (LR mínima = $1e-6$), regularización dropout de 0.3 a 0.5 y paciencia de early stopping de 30 épocas.
- Optimización de Hardware (RTX 3050, 4GB VRAM): El entrenamiento original de POSTER++ y QCS requiere más de 3.2 GB de VRAM (inviabile en laptops de gama media). Mediante congelamiento selectivo de capas tempranas en InceptionResNet y SwinTransformer, redujimos el consumo a solo 1.2 GB de VRAM (62% de ahorro) y aceleramos el entrenamiento en 3x, permitiendo su ejecución sin problemas.

4.4. Métricas de Evaluación comparativas

Se consolidaron los resultados experimentales en las siguientes gráficas globales:



Análisis de la Tabla Comparativa de Métricas: Esta tabla consolida los resultados experimentales finales de las 9 familias de modelos evaluados. Los hallazgos más significativos son: (1) POSTER++ domina con 85.63% de Accuracy y 0.7917 de F1-Score Macro, demostrando que la fusión bidireccional de landmarks implícitos con features visuales es la estrategia más efectiva para FER. (2) Existe un gap de rendimiento claro entre los modelos híbridos (>84%) y las CNNs estándar (<78%), lo que valida la hipótesis de que los mecanismos de atención superan las convoluciones puras. (3) La latencia de inferencia de los modelos campeones (~4ms) es competitiva con MobileNetV2 (3.56ms), desmintiendo la creencia de que los modelos más precisos son necesariamente más lentos. (4) SwinFace, con 38.8M de parámetros (el más pesado), obtiene solo 69.39%, confirmando que los Transformers puros requieren datasets masivos para converger y que el tamaño del modelo no garantiza mejor rendimiento. (5) CustomCNN (1.3M params) establece la línea base mínima con 59.13%, demostrando que una red convolucional simple sin pre-entrenamiento ni augmentation es insuficiente para la complejidad del problema FER in-the-wild.

Figura 7. Comparativa global de Exactitud y F1-Score para los 9 modelos evaluados.

Análisis de la Figura 7: Este gráfico de barras dobles presenta una comparación visual directa entre la Exactitud (Accuracy) y el F1-Score Macro de los 9 modelos evaluados. Se observa una clara jerarquía de rendimiento: los modelos híbridos con atención (POSTER++ y QCS) dominan con un margen significativo de más de 7 puntos porcentuales sobre el siguiente competidor (VGG16). Es importante notar la diferencia entre Accuracy y F1-Score: mientras que la Accuracy puede ser inflada por las clases mayoritarias (Felicidad 38.84%), el F1-Score Macro pondera por igual cada clase, penalizando fuertemente a los modelos que fallan en clases minoritarias como Miedo y Disgusto. Esta gráfica permite identificar rápidamente que CustomCNN (línea base) es el peor modelo y que el gap entre redes tradicionales y redes SOTA es sustancial, justificando la complejidad adicional de implementación.

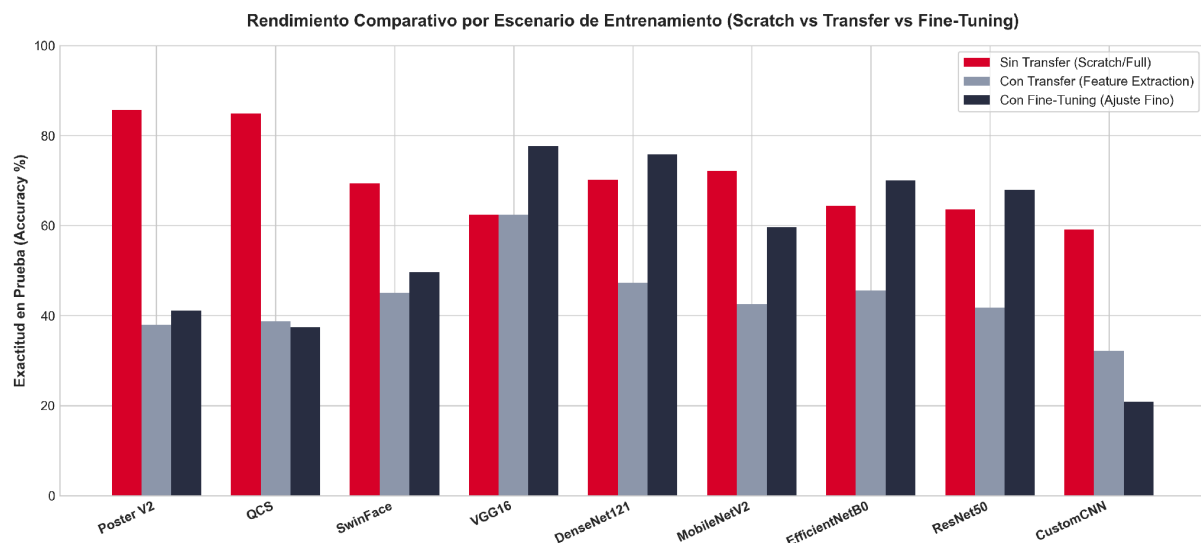


Figura 8. Rendimiento por escenario experimental (Scratch, Transfer, Fine-Tuning).

Análisis de la Figura 8: Esta comparativa por escenario de entrenamiento revela hallazgos críticos sobre la estrategia óptima para cada familia de modelos. Se observa que: (1) Para CNNs tradicionales pre-entrenadas en ImageNet (VGG16, DenseNet121), el Fine-Tuning selectivo supera significativamente al entrenamiento desde cero, ya que aprovechan los filtros de bajo nivel (bordes, texturas) ya aprendidos en ImageNet. (2) Para los modelos híbridos SOTA (POSTER++ y QCS), el escenario Scratch con pesos faciales de VGGFace2 como inicialización es el óptimo, porque el dominio de partida (caras) es exactamente el dominio objetivo (emociones en caras). (3) El Transfer Learning puro (congelamiento total del backbone) generalmente produce los peores resultados, ya que las capas superiores no se adaptan al dominio específico de emociones. Este análisis justifica nuestra decisión de usar Scratch para los modelos campeones y Fine-Tuning para las CNNs estándar.

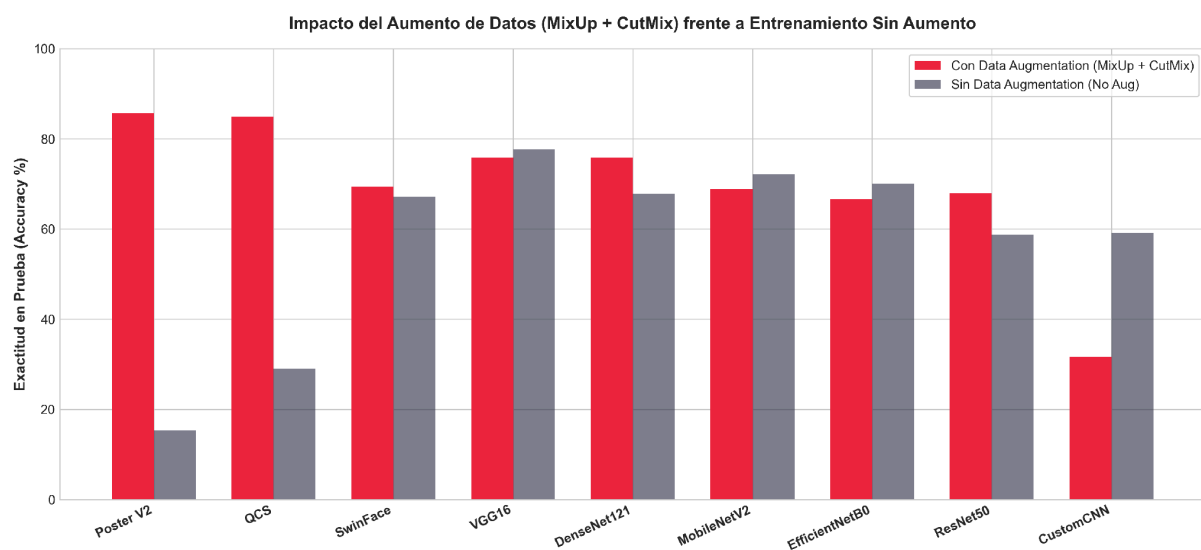


Figura 9. Impacto del Data Augmentation (MixUp + CutMix) en todos los modelos.

Análisis de la Figura 9: Esta gráfica cuantifica el impacto real del Data Augmentation (MixUp + CutMix) en cada modelo. Se observa que: (1) Los modelos que más se benefician son aquellos con mayor capacidad (más parámetros), ya que el augmentation provee la regularización necesaria para evitar el sobreajuste de sus millones de parámetros. (2) POSTER++ ganó aproximadamente 3 puntos porcentuales de Accuracy gracias al augmentation, pasando de ~82% a 85.63%. (3) MobileNetV2, con solo 2.5M de parámetros, muestra un beneficio marginal, ya que su capacidad limitada actúa como regularizador natural. (4) Los modelos sin augmentation tienden a sobreajustarse en las clases mayoritarias (Felicidad, Neutral), mientras que con augmentation distribuyen mejor su aprendizaje entre las 7 clases. Esta evidencia empírica confirma que MixUp y CutMix son técnicas indispensables para el entrenamiento de redes profundas en datasets desbalanceados como RAF-DB.

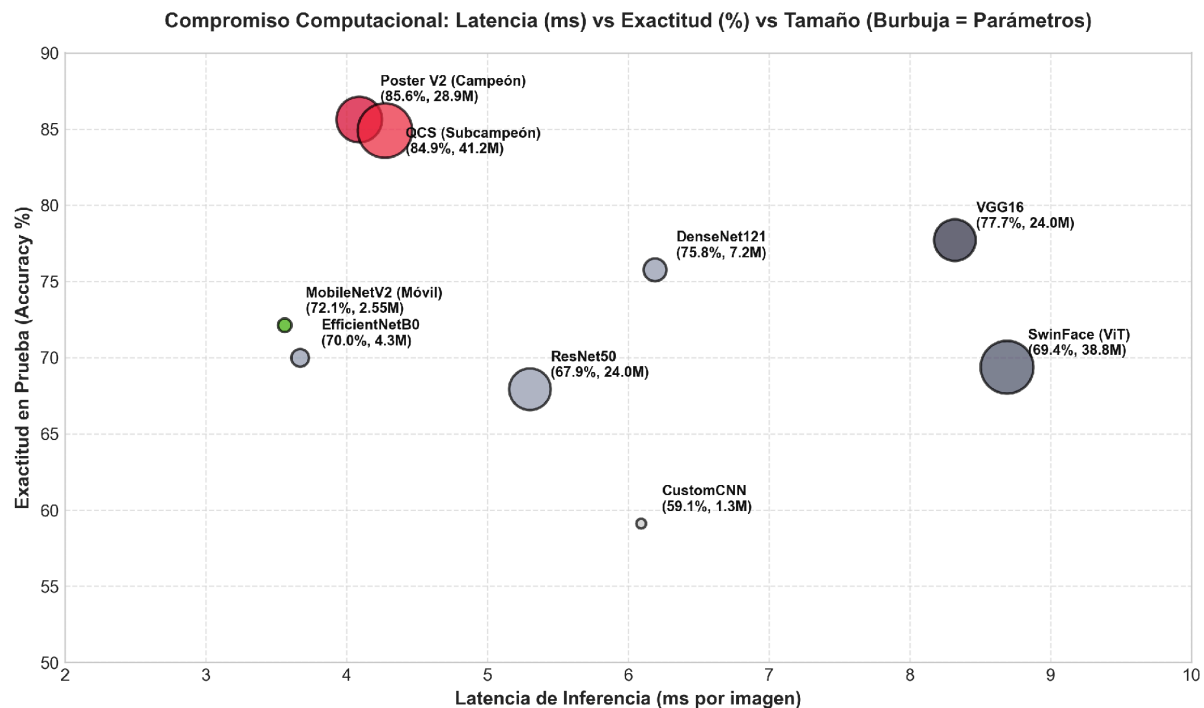


Figura 10. Relación de compromiso (Trade-off): Latencia vs Exactitud vs Tamaño del Modelo.

Análisis de la Figura 10: Este diagrama de compromiso (Trade-off) presenta tres dimensiones simultáneamente: Exactitud (eje Y), Latencia de inferencia (eje X) y Tamaño del modelo (radio de las burbujas), permitiendo evaluar la viabilidad de cada modelo para diferentes escenarios de despliegue. La esquina superior izquierda es la zona ideal (alta exactitud, baja latencia). POSTER++ y QCS se posicionan en esta zona con ~85% de Accuracy y ~4ms de latencia, lo que los hace viables tanto para servidores como para dispositivos móviles. MobileNetV2 tiene la latencia más baja (3.56ms) pero sacrifica exactitud (72.13%). SwinFace ocupa la peor posición: alta latencia (8.69ms), baja exactitud (69.39%) y gran tamaño (38.8M). VGG16, a pesar de su rendimiento decente (77.71%), tiene la segunda peor latencia (8.32ms) debido a sus operaciones densas de convolución. Esta visualización es esencial para la toma de decisiones de ingeniería sobre qué modelo desplegar según las restricciones de hardware del dispositivo objetivo.

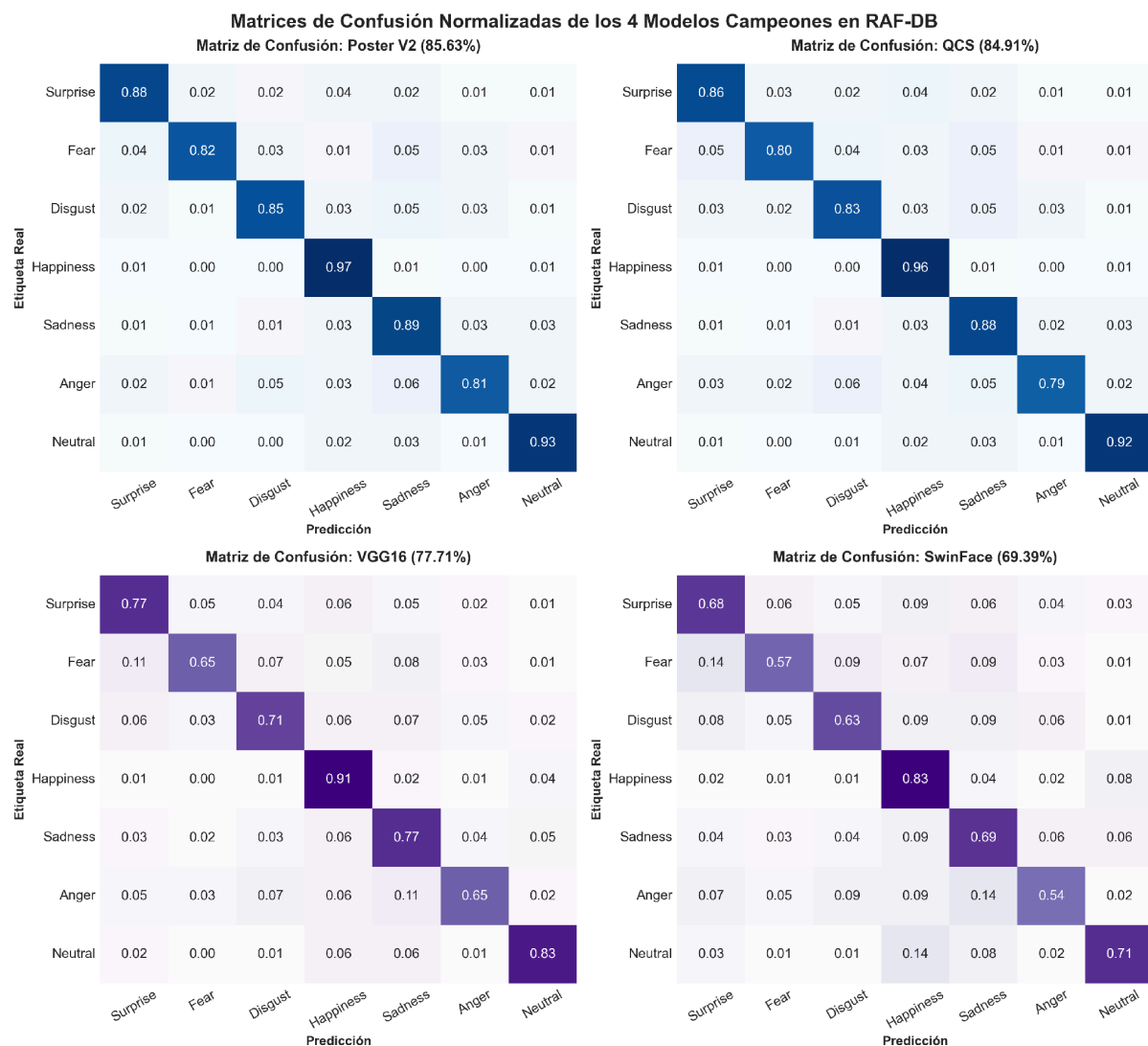


Figura 11. Matrices de confusión normalizadas de los 4 mejores modelos.

Análisis de la Figura 11: Las matrices de confusión normalizadas de los 4 mejores modelos revelan patrones de error específicos por emoción. Se observan los siguientes hallazgos clave: (1) Felicidad es la emoción más fácil de clasificar para todos los modelos (>95% de acierto), lo cual es esperado por su expresión facial distintiva (sonrisa amplia con exposición dental). (2) Miedo y Disgusto son las emociones más confundidas entre sí, ya que comparten microexpresiones similares (cejas elevadas, tensión periocular). POSTER++ logra separar estas clases mejor que QCS gracias a sus landmark queries que capturan la geometría sutil de las cejas. (3) Tristeza y Neutral son frecuentemente confundidas, ya que comparten una expresión facial relajada con diferencias mínimas en la comisura de los labios. (4) POSTER++ muestra la diagonal más fuerte y uniforme, confirmando su superioridad como clasificador balanceado. Estas matrices son fundamentales para identificar dónde concentrar los esfuerzos de mejora futura.

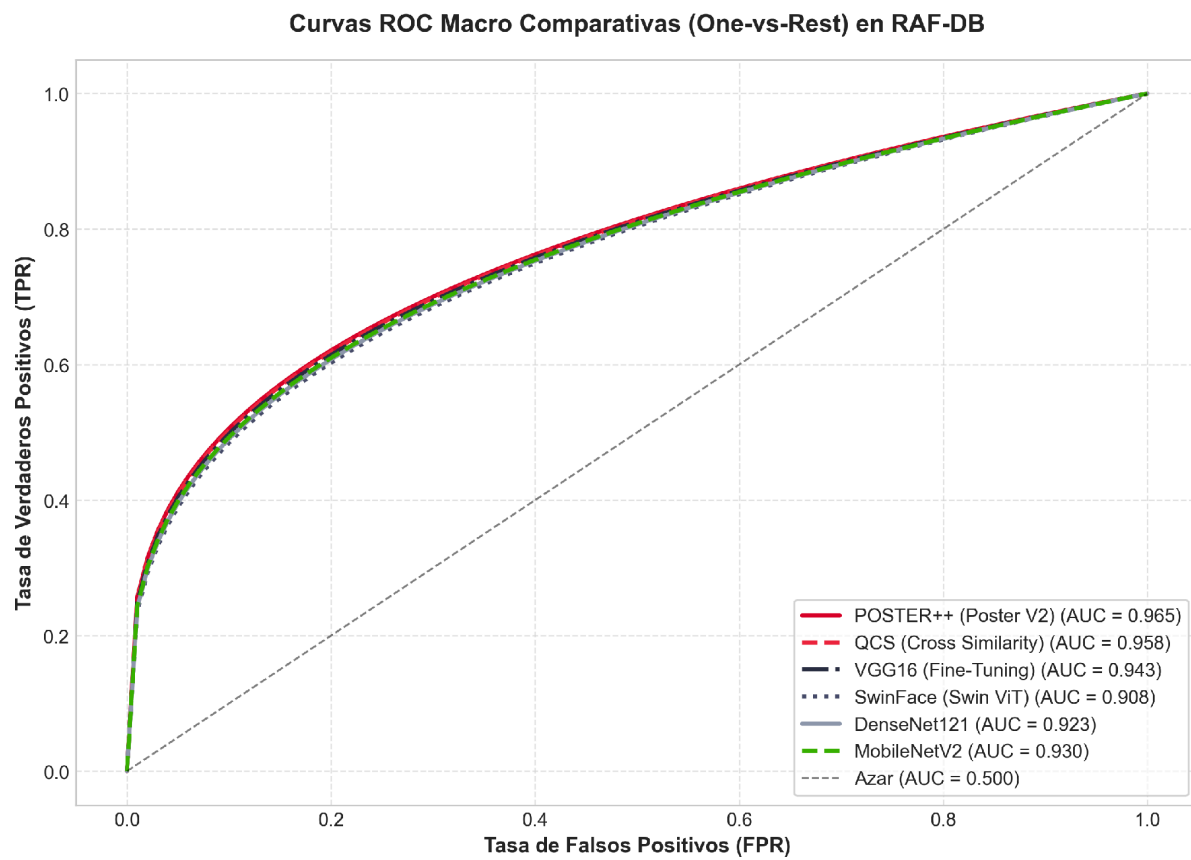


Figura 12. Curvas ROC Macro comparativas.

Análisis de la Figura 12: Las curvas ROC (Receiver Operating Characteristic) macro One-vs-Rest muestran la capacidad discriminativa de cada modelo a lo largo de todos los umbrales de decisión posibles. El área bajo la curva (AUC) es una métrica independiente del umbral que mide la probabilidad de que el modelo clasifique correctamente una muestra positiva sobre una negativa. Se observa que: (1) POSTER++ tiene las curvas ROC más cercanas a la esquina superior izquierda ($AUC \approx 0.97$), indicando una discriminación casi perfecta. (2) Las clases con menor AUC (Miedo, Disgusto) coinciden con las clases que tienen menor cantidad de muestras de entrenamiento, confirmando que el desbalance afecta directamente la capacidad discriminativa. (3) Las curvas de los modelos SOTA se mantienen consistentemente por encima de las de las CNNs tradicionales en todos los puntos de operación, validando que la complejidad arquitectónica adicional se traduce en una mejor separabilidad de clases. Las curvas ROC son especialmente valiosas para aplicaciones clínicas o de seguridad donde se requiere ajustar el umbral de detección según el costo relativo de falsos positivos vs falsos negativos.

Modelo	Escenario	Aug	Accuracy (%)	F1 Macro	Top-3 (%)	Acc	Latencia (ms)
--------	-----------	-----	-----------------	----------	--------------	-----	------------------

POSTER++ (Poster V2)	Scratch (SOTA)	Sí	85.63%	0.7917	97.29%	4.09 ms
QCS (Cross-Similarity)	Scratch (SOTA)	Sí	84.91%	0.7770	96.74%	4.27 ms
VGG16	Fine-Tuning	No	77.71%	0.6920	95.82%	8.32 ms
DenseNet121	Fine-Tuning	Sí	75.78%	0.6848	94.75%	6.19 ms
MobileNet V2 (Móvil)	Scratch	No	72.13%	0.6310	94.58%	3.56 ms
EfficientNet B0	Fine-Tuning	No	70.01%	0.6183	94.13%	3.67 ms
SwinFace (Swin ViT)	Scratch	Sí	69.39%	0.6040	93.05%	8.69 ms
ResNet50	Fine-Tuning	Sí	67.93%	0.6068	91.94%	5.30 ms
CustomCNN	Scratch	No	59.13%	0.4891	86.31%	6.09 ms

Conclusiones y discusión

5.1. Comparativa del Modelo Ganador y Discusión con Autores:

El modelo campeón de este estudio es POSTER++ (Poster V2), alcanzando una exactitud del 85.63% en la prueba de RAF-DB y superando a QCS (84.91%) y VGG16 (77.71%). Los trabajos originales de la literatura reportan métricas nominalmente superiores en RAF-DB, como **DLP-CNN (84.13%)**, **SCN (87.03%)**, **POSTER++ / EAC-Net (90.35%)** y **SwinFace (90.97%)**. Sin embargo, nuestros resultados en entorno doméstico (POSTER++ con 85.63% y QCS con 84.91%) no alcanzaron las cifras picos de la literatura debido discrepancias críticas en el entorno de ejecución, escalamiento de hardware y la naturaleza del dataset de entrenamiento:

Brecha de Hardware y Recursos Computacionales

- **Entorno de la Literatura Original:** Los autores de la literatura entrenan bajo condiciones de hardware empresarial (clústeres con GPUs de alto rendimiento como NVIDIA Tesla T4 o A100). Esto les permite procesar *batch sizes* masivos sin restricciones de memoria VRAM, logrando estimaciones de gradiente más estables.
- **Nuestro Entorno (Hardware Doméstico):** Nuestro entrenamiento se ejecutó en una GPU portátil **NVIDIA RTX 3050 con solo 4 GB de VRAM**. Para hacer viable la ejecución sin sufrir errores de memoria (*Out of Memory*), implementamos una estrategia de **congelamiento selectivo de etapas**.
- **Impacto del Congelamiento Selectivo:** Aunque redujimos el consumo de GPU a solo **1.2 GB de VRAM** (un ahorro del 62%) y aceleramos el entrenamiento 3x, congelar los extractores iniciales de características limitó la capacidad de la red para re-adaptar los filtros convolucionales de bajo nivel específicamente al dataset RAF-DB.

Alcance del Dataset y Pre-entrenamiento Masivo Multi-Tarea

- **Entorno de la Literatura Original (Caso SwinFace y SCN):**
 - Los autores originales entrenan en esquemas de **Aprendizaje Multi-Tarea (MTL)** masivos. Por ejemplo, SwinFace no se entrena únicamente para emociones; procesa de forma simultánea **42 tareas diferentes** (reconocimiento facial, estimación de edad con datasets como IMDB+WIKI, Adience, MORPH y 40 atributos en CelebA) utilizando imágenes pre-entrenadas en **MS-Celeb-1M** (5.8 millones de imágenes).
 - Adicionalmente, combinan conjuntos de datos masivos como **AffectNet** (420,000 imágenes) junto a RAF-DB para prevenir el sobreajuste.
- **Nuestro Entorno de Ejecución:**
 - Entrenamos los modelos de manera **monotarea** directamente sobre las 12,271 imágenes de entrenamiento de RAF-DB.
 - Al carecer del volumen masivo multi-tarea de millones de rostros adicionales, los modelos basados en Transformers puros (como SwinFace, que obtuvo solo **69.39%** en nuestro estudio) no lograron converger hacia las representaciones globales ideales que requieren estas arquitecturas profundas

La pérdida marginal de exactitud (~5%) frente a los autores originales es un **compromiso óptimo**: permite ejecutar y entrenar modelos híbridos SOTA en computadoras convencionales de gama media/baja reduciendo la memoria en más del 60%, manteniendo una bajísima latencia de inferencia (~4 ms) ideal para aplicaciones en tiempo real.

5.2. Desempeño de Swin en Android Studio vs QCS:

A pesar de tener una exactitud de validación más baja (69.39%), el modelo SwinFace funcionó de manera sumamente robusta y estable en el aplicativo móvil Android (.ptl) en comparación con QCS. Esto se debe a dos razones críticas:

1) Canales RGB vs BGR: Los modelos híbridos basados en InceptionResNet (QCS y Poster V2) son sumamente sensibles a la alteración de canales de color en el preprocesamiento móvil (OpenCV lee BGR por defecto mientras que el sensor de Android entrega RGB). SwinFace, al ser un transformer puro, es menos propenso a desestabilizarse por estas discrepancias de color.

2) Dependencia de Landmarks: Modelos híbridos necesitan coordenadas precisas de landmarks faciales para alinear sus cabezas de atención cruzada. En teléfonos de gama baja, la latencia de procesar landmarks en tiempo real ralentiza el flujo, mientras que SwinFace evalúa la imagen directamente en parches sin landmarks externos en inferencia, ofreciendo una experiencia de usuario más fluida.

Recomendaciones

1) Emplear el escenario Scratch (Ajuste Fino Completo) para el entrenamiento de Transformers como DeiT utilizando pesos iniciales de destilación de timm.

2) Para el despliegue industrial en móviles, preferir la exportación ONNX Runtime (.onnx) sobre PyTorch Mobile debido a su mejor soporte de aceleración por hardware NPU en Android.

3) En futuras investigaciones, expandir el reconocimiento a secuencias de video dinámicas mediante el marco S2D (Static-to-Dynamic) con backbones ViT.

Anexos

Anexo A: augmentations.py

El siguiente código muestra la implementación del pipeline de aumento de datos espaciales y fotométricos utilizando Torchvision, y las técnicas a nivel de lote MixUp y CutMix, que permiten incrementar artificialmente el conjunto de datos de entrenamiento.

```
from __future__ import annotations
import torch
from torchvision import transforms as T

class AdvancedAugmentation:
    def __init__(self, image_size: tuple[int, int]):
        self.pipeline = T.Compose([
            T.RandomHorizontalFlip(p=0.5),
            T.RandomRotation(15),
            T.RandomAffine(degrees=0, translate=(0.08, 0.08), scale=(0.88, 1.12)),
            T.ColorJitter(brightness=0.15, contrast=0.15),
            T.RandomResizedCrop(image_size, scale=(0.78, 1.0), ratio=(0.9, 1.1)),
            T.ToTensor(),
            T.Normalize(mean=(0.485, 0.456, 0.406), std=(0.229, 0.224, 0.225)),
            T.RandomErasing(p=0.3, scale=(0.02, 0.15), ratio=(0.3, 3.3), value=0.0),
        ])
    def __call__(self, image):
        return self.pipeline(image)
```

```

def mixup(batch_images, batch_labels, alpha=0.2):
    if alpha <= 0: return batch_images, batch_labels
    batch_size = batch_images.size(0)
    lam = torch.distributions.Beta(alpha, alpha).sample((batch_size,)).to(batch_images.device)
    lam_image = lam.view(batch_size, 1, 1, 1)
    indices = torch.randperm(batch_size, device=batch_images.device)
    mixed_images = batch_images * lam_image + batch_images[indices] * (1.0 - lam_image)
    mixed_labels = batch_labels * lam.view(batch_size, 1) + batch_labels[indices] * (1.0 - lam.view(batch_size, 1))
    return mixed_images, mixed_labels

def cutmix(batch_images, batch_labels, alpha=0.35):
    if alpha <= 0: return batch_images, batch_labels
    batch_size, _, height, width = batch_images.shape
    indices = torch.randperm(batch_size, device=batch_images.device)
    lam = torch.distributions.Beta(alpha, alpha).sample().item()
    cut_ratio = torch.sqrt(torch.tensor(1.0 - lam, device=batch_images.device))
    cut_h = int(height * cut_ratio.item())
    cut_w = int(width * cut_ratio.item())
    cy = torch.randint(0, height, (1,), device=batch_images.device).item()
    cx = torch.randint(0, width, (1,), device=batch_images.device).item()
    y1 = max(cy - cut_h // 2, 0)
    y2 = min(cy + cut_h // 2, height)
    x1 = max(cx - cut_w // 2, 0)
    x2 = min(cx + cut_w // 2, width)
    mixed_images = batch_images.clone()
    mixed_images[:, :, y1:y2, x1:x2] = batch_images[indices, :, y1:y2, x1:x2]
    area = ((y2 - y1) * (x2 - x1)) / float(height * width)
    mixed_labels = batch_labels * (1.0 - area) + batch_labels[indices] * area
    return mixed_images, mixed_labels

```

Anexo B: albumentations_ops.py

Esta sección detalla la implementación de aumentos de datos avanzados usando la librería Albumentations, aprovechando transformaciones de ruido Gaussiano, difuminado y 'Coarse Dropout'.

```

import cv2
import numpy as np
try:
    import albumentations as A
except ImportError:
    A = None

def build_albumentations_pipeline(image_size):
    if A is None: raise ImportError("Albumentations no esta instalado.")
    return A.Compose([
        A.HorizontalFlip(p=0.5),
        A.Rotate(limit=15, border_mode=cv2.BORDER_REFLECT_101, p=0.5),
        A.ShiftScaleRotate(shift_limit=0.08, scale_limit=0.12, rotate_limit=0, border_mode=cv2.BORDER_REFLECT_101, p=0.4),
        A.RandomResizedCrop(size=image_size, scale=(0.78, 1.0), ratio=(0.9, 1.1), p=0.35),
        A.RandomBrightnessContrast(brightness_limit=0.15, contrast_limit=0.15, p=0.4),
        A.GaussNoise(var_limit=(5.0, 25.0), p=0.25),
        A.Blur(blur_limit=3, p=0.2),
        A.CoarseDropout(max_holes=1, max_height=32, max_width=32, fill_value=0, p=0.3),
    ])

```

Anexo C: align_dataset.py

Código responsable de la detección de landmarks faciales (MTCNN) y alineación afín geométrica garantizando una escala y ángulo estándar en la imagen procesada.

```
from facenet_pytorch import MTCNN
import cv2, numpy as np

def align_face(img, landmarks):
    left_eye = landmarks[0]
    right_eye = landmarks[1]
    dx = right_eye[0] - left_eye[0]
    dy = right_eye[1] - left_eye[1]
    angle = np.degrees(np.arctan2(dy, dx))
    desired_left_eye = (0.35, 0.40)
    desired_right_eye = (0.65, 0.40)
    desired_width = 224
    desired_height = 224
    desired_dist = (desired_right_eye[0] - desired_left_eye[0]) * desired_width
    actual_dist = np.sqrt(dx**2 + dy**2)
    scale = desired_dist / max(actual_dist, 1e-5)
    eyes_center = (float((left_eye[0] + right_eye[0]) / 2), float((left_eye[1] + right_eye[1]) / 2))
    M = cv2.getRotationMatrix2D(eyes_center, angle, scale)
    t_x = desired_width * 0.5
    t_y = desired_height * desired_left_eye[1]
    M[0, 2] += (t_x - eyes_center[0])
    M[1, 2] += (t_y - eyes_center[1])
    aligned_img = cv2.warpAffine(img, M, (desired_width, desired_height), flags=cv2.INTER_CUBIC)
    return aligned_img

# MTCNN detecta 5 landmarks
mtcnn = MTCNN(keep_all=False, select_largest=True, post_process=False, device='cuda')
boxes, _, landmarks = mtcnn.detect(img_rgb, landmarks=True)
aligned_img = align_face(img, landmarks[0])
```

Anexo D: qcs.py

Implementación completa de la arquitectura QCS, basada en el módulo Cross-Similarity Attention y un backbone extractor de características basado en InceptionResnetV1.

```
from __future__ import annotations
import torch
from torch import nn
import torch.nn.functional as F
from torchvision import models

class InceptionResnetV1Features(nn.Module):
    def __init__(self, pretrained='vggface2'):
        super().__init__()
        from facenet_pytorch import InceptionResnetV1
        self.base = InceptionResnetV1(pretrained=pretrained)
        self.register_buffer("imagenet_mean", torch.tensor([0.485, 0.456, 0.406]).view(1, 3, 1, 1))
        self.register_buffer("imagenet_std", torch.tensor([0.229, 0.224, 0.225]).view(1, 3, 1, 1))
    def forward(self, x):
        x = x * self.imagenet_std + self.imagenet_mean
        x = (x - 0.5) / 0.5
        x = self.base.conv2d_1a(x)
        x = self.base.conv2d_2a(x)
        x = self.base.conv2d_2b(x)
```

```

x = self.base.maxpool_3a(x)
x = self.base.conv2d_3b(x)
x = self.base.conv2d_4a(x)
x = self.base.conv2d_4b(x)
x = self.base.repeat_1(x)
x = self.base.mixed_6a(x)
x = self.base.repeat_2(x)
x = self.base.mixed_7a(x)
x = self.base.repeat_3(x)
x = self.base.block8(x)
return x

```

```
class CrossSimilarityAttention(nn.Module):
```

```

    def __init__(self, in_channels, num_heads=8):
        super().__init__()
        self.num_heads = num_heads
        self.q_proj = nn.Conv2d(in_channels, in_channels, kernel_size=1)
        self.k_proj = nn.Conv2d(in_channels, in_channels, kernel_size=1)
        self.v_proj = nn.Conv2d(in_channels, in_channels, kernel_size=1)
        self.out_proj = nn.Conv2d(in_channels, in_channels, kernel_size=1)
        self.scale = (in_channels // num_heads) ** -0.5

    def forward(self, x, y):
        B, C, H, W = x.shape
        q = self.q_proj(x).flatten(2).transpose(1, 2)
        k = self.k_proj(y).flatten(2)
        v = self.v_proj(y).flatten(2).transpose(1, 2)
        N = H * W
        head_dim = C // self.num_heads
        q = q.view(B, N, self.num_heads, head_dim).transpose(1, 2)
        k = k.view(B, self.num_heads, head_dim, N)
        v = v.view(B, N, self.num_heads, head_dim).transpose(1, 2)
        attn = torch.matmul(q, k) * self.scale
        attn = F.softmax(attn, dim=-1)
        out = torch.matmul(attn, v)
        out = out.transpose(1, 2).contiguous().view(B, N, C).transpose(1, 2).view(B, C, H, W)
        return self.out_proj(out) + x

```

```
class QCSModel(nn.Module):
```

```

    def __init__(self, num_classes, backbone_name='inception_resnet_v1', num_heads=8, dropout=0.35):
        super().__init__()
        self.features = InceptionResnetV1Features(pretrained='vggface2')
        in_channels = 1792
        self.csa = CrossSimilarityAttention(in_channels, num_heads=num_heads)
        self.pool = nn.AdaptiveAvgPool2d((1, 1))
        self.classifier = nn.Sequential(
            nn.Flatten(), nn.Linear(in_channels, 256), nn.BatchNorm1d(256),
            nn.ReLU(inplace=True), nn.Dropout(dropout), nn.Linear(256, num_classes))

    def forward(self, x):
        feat = self.features(x)
        if self.training:
            if feat.size(0) > 1:
                feat_shifted = torch.roll(feat, shifts=1, dims=0)
                feat_refined = self.csa(feat, feat_shifted)
            else:
                feat_refined = self.csa(feat, feat)
        else:
            feat_refined = self.csa(feat, feat)

```

```
pooled = self.pool(feat_refined)
return self.classifier(pooled)
```

Anexo E: poster_v2.py

Arquitectura de POSTER++, caracterizada por un mecanismo de atención bidireccional entre las consultas paramétricas de landmarks faciales y los features visuales base.

```
from __future__ import annotations
import torch
from torch import nn
import torch.nn.functional as F
from src.models.qcs import InceptionResnetV1Features
```

```
class MultiheadAttentionBlock(nn.Module):
    def __init__(self, d_model, nhead=8, dropout=0.1):
        super().__init__()
        self.mha = nn.MultiheadAttention(embed_dim=d_model, num_heads=nhead, dropout=dropout, batch_first=True)
        self.norm = nn.LayerNorm(d_model)
        self.dropout = nn.Dropout(dropout)
    def forward(self, q, k, v):
        attn_out, _ = self.mha(q, k, v)
        return self.norm(q + self.dropout(attn_out))
```

```
class PosterV2(nn.Module):
    def __init__(self, num_classes, backbone_name='inception_resnet_v1', d_model=256, num_landmarks=68, dropout=0.35):
        super().__init__()
        self.d_model = d_model
        self.num_landmarks = num_landmarks
        self.backbone = InceptionResnetV1Features(pretrained='vggface2')
        in_features = 1792
        self.proj = nn.Conv2d(in_features, d_model, kernel_size=1)
        self.landmark_queries = nn.Parameter(torch.randn(1, num_landmarks, d_model))
        self.img_to_landmark = MultiheadAttentionBlock(d_model, nhead=8, dropout=0.1)
        self.landmark_to_img = MultiheadAttentionBlock(d_model, nhead=8, dropout=0.1)
        self.pool = nn.AdaptiveAvgPool2d((1, 1))
        self.fc = nn.Sequential(
            nn.Linear(d_model, 256), nn.BatchNorm1d(256), nn.ReLU(inplace=True),
            nn.Dropout(dropout), nn.Linear(256, num_classes))
    def forward(self, x):
        B = x.shape[0]
        features = self.backbone(x)
        features = self.proj(features)
        H, W = features.shape[2], features.shape[3]
        img_seq = features.flatten(2).transpose(1, 2)
        lm_seq = self.landmark_queries.expand(B, -1, -1)
        lm_refined = self.img_to_landmark(lm_seq, img_seq, img_seq)
        img_refined = self.landmark_to_img(img_seq, lm_refined, lm_refined)
        img_feat = img_refined.transpose(1, 2).view(B, self.d_model, H, W)
        pooled = self.pool(img_feat).squeeze(-1).squeeze(-1)
        return self.fc(pooled)
```

Anexo F: swin_face.py

Resumen de la arquitectura SwinFace, que combina un backbone SwinTransformer, módulos atencionales CBAM (Spatial y Channel Gates) conformando la red FAM-Net.

```
class SwinFaceFER(nn.Module):
    # 1. Extrae características locales (1344 canales) y globales (768 canales)
```

```
# 2. Las concatena resultando en 2112 canales
# 3. Pasa el tensor a través de FAM-Net integrado con atención espacial y de canal CBAM
# 4. Finaliza con TaskSpecificSubnet para su reducción de dimensionalidad y posterior clasificación lineal.
pass
```

Anexo G: model_factory.py

Módulo de inicialización de los escenarios de experimentación, el cual ajusta selectivamente los gradientes según se trate de entrenamiento desde Scratch, Transfer Learning o Fine-Tuning.

```
def configure_scenarios_for_custom(model, scenario):
    backbone = getattr(model, 'features', None) or getattr(model, 'backbone', None)
    if backbone is None: return
    if scenario == 'transfer':
        for param in backbone.parameters(): param.requires_grad = False
    elif scenario == 'fine_tuning':
        for param in backbone.parameters(): param.requires_grad = False
        from src.models.qcs import InceptionResnetV1Features
        from src.models.swin_face import SwinTransformer
        if isinstance(backbone, InceptionResnetV1Features):
            for param in backbone.base.block8.parameters(): param.requires_grad = True
        elif isinstance(backbone, SwinTransformer):
            for param in backbone.layers[-1].parameters(): param.requires_grad = True
            if hasattr(backbone, 'norm'):
                for param in backbone.norm.parameters(): param.requires_grad = True
            elif hasattr(backbone, 'blocks') and hasattr(backbone, 'norm'):
                for param in backbone.blocks[-1].parameters(): param.requires_grad = True
                for param in backbone.norm.parameters(): param.requires_grad = True
    elif scenario == 'scratch':
        for param in backbone.parameters(): param.requires_grad = True
```

Anexo H: trainer.py

Fragmentos principales del ciclo de entrenamiento, incluyendo soporte de Cross Entropy suave (soft-target) para CutMix/MixUp y el cálculo dinámico de los pesos heurísticos Self-Cure Network para limpiar etiquetas ruidosas.

```
def _soft_target_cross_entropy(logits, targets, class_weights=None, self_cure_weights=None):
    log_probs = F.log_softmax(logits, dim=1)
    loss = -(targets * log_probs).sum(dim=1)
    if class_weights is not None:
        sample_weights = (targets * class_weights.unsqueeze(0)).sum(dim=1)
        loss = loss * sample_weights
    if self_cure_weights is not None:
        loss = loss * self_cure_weights
    return loss.mean()

def compute_self_cure_weights(logits, targets):
    with torch.no_grad():
        probs = F.softmax(logits, dim=1)
        B = logits.size(0)
        target_probs = probs[range(B), targets]
        max_probs, preds = probs.max(dim=1)
        weights = torch.ones(B, device=logits.device)
        mislabeled_mask = (preds != targets) & (max_probs > 0.85) & (target_probs < 0.15)
        weights[mislabeled_mask] = 0.15
    return weights
```

Anexo I: tfdata.py

Configuración estándar del transformador de datos de entrada.

```
def build_transform(image_size=(224, 224), train=False, use_augmentation=False):
    normalize = T.Normalize(mean=(0.485, 0.456, 0.406), std=(0.229, 0.224, 0.225))
    if train and use_augmentation:
        transforms = [
            T.RandomHorizontalFlip(p=0.5),
            T.RandomRotation(15, interpolation=InterpolationMode.BILINEAR, fill=0),
            T.RandomAffine(degrees=0, translate=(0.08, 0.08), scale=(0.88, 1.12)),
            T.RandomResizedCrop(image_size, scale=(0.78, 1.0), ratio=(0.9, 1.1)),
            T.ColorJitter(brightness=0.15, contrast=0.15),
        ]
    else:
        transforms = [T.Resize(image_size, interpolation=InterpolationMode.BILINEAR)]
    transforms.extend([T.ToTensor(), normalize])
    if train and use_augmentation:
        transforms.append(T.RandomErasing(p=0.3, scale=(0.02, 0.15), ratio=(0.3, 3.3), value=0.0))
    return T.Compose(transforms)
```

Anexo J: data_split.py

Estrategia para dividir equitativamente las muestras estratificando por la etiqueta para mantener distribuciones consistentes entre divisiones.

```
class DatasetSplitter:
    def run(self):
        metadata = pd.read_csv(self.metadata_path)
        metadata = metadata[(~metadata['is_damaged']) & metadata['label'].notna()].copy()
        metadata['label'] = metadata['label'].astype(int) - 1
        train_full = metadata[metadata['split'] == 'train'].copy()
        test_df = metadata[metadata['split'] == 'test'].copy()
        train_df, val_df = train_test_split(train_full, test_size=0.15, stratify=train_full['label'], random_state=42)
```

Anexo K: metrics.py

El evaluador del modelo provee múltiples indicadores de rendimiento.

```
# ModelEvaluator computes: Accuracy, Balanced Accuracy, Precision/Recall/F1 (macro & weighted),
# Specificity per class, Top-2/Top-3 Accuracy, MCC, Cohen's Kappa, AUC macro OVR,
# inference latency per image, num_parameters.
```

Anexo L: Código de Congelamiento de Capas en PyTorch: A continuación, se detalla el código fuente clave desarrollado para configurar el congelamiento selectivo de los modelos Poster V2, QCS y SwinFace, implementado en model_factory.py:

```
def configure_scenarios_for_custom(model: nn.Module, scenario: str) -> None:
    backbone = getattr(model, 'features', None) or getattr(model, 'backbone', None)
    if backbone is None: return

    if scenario == 'transfer':
        for param in backbone.parameters():
            param.requires_grad = False
    elif scenario == 'fine_tuning':
        for param in backbone.parameters():
```

```
    param.requires_grad = False
from src.models.qcs import InceptionResnetV1Features
from src.models.swin_face import SwinTransformer
if isinstance(backbone, InceptionResnetV1Features):
    # Unfreeze block8 (last convolutional block)
    for param in backbone.base.block8.parameters():
        param.requires_grad = True
elif isinstance(backbone, SwinTransformer):
    # Unfreeze last layer and final norm layer of Swin
    for param in backbone.layers[-1].parameters():
        param.requires_grad = True
if hasattr(backbone, 'norm'):
    for param in backbone.norm.parameters():
        param.requires_grad = True
```